



FACULTAD DE
INGENIERIA
UNIVERSIDAD NACIONAL DE JUJUY

IrisDual 3D — Backend API

Trabajo Práctico Integrador (TPI) — Analista Programador Universitario

Materia	Programación Orientada a Objetos — POO
Institución	Facultad de Ingeniería — Universidad Nacional de Jujuy (UNJu)
Docente	Mg. Ing. Ariel Alejandro Vega

Alumno	Cussi Walter Leonel
DNI	45.880.218
Libreta Universitaria	APU006422

Repositorio GitHub	github.com/EnderJack379/irisdual3d-pedidos-api
Período	Septiembre 2026
Artefacto Maven	com.irisdual / pedidos-api / 0.0.1-SNAPSHOT



1. Resumen y Objetivos del Proyecto

IrisDual 3D Backend API es una solución de software orientada a servicios REST, desarrollada con **Spring Boot 3.2.2** sobre **Java 17 LTS**, destinada a gestionar integralmente las operaciones de un negocio de impresión 3D bajo demanda. El sistema provee un backend seguro, escalable y desacoplado para la administración de catálogos de piezas, categorías y pedidos de clientes.

La arquitectura implementada sigue el paradigma **multicapa** (Layered Architecture), garantizando la separación de responsabilidades entre la presentación, la lógica de negocio y el acceso a datos. La persistencia se resuelve mediante **Spring Data JPA / Hibernate** sobre **MySQL 8.x**, con mapeo objeto-relacional y generación de esquema mediante **ddl-auto=update**.

La seguridad de los endpoints está garantizada por Spring Security 6 con autenticación stateless basada en JSON Web Tokens (JWT), algoritmo HS256, expiración de 24 horas. Solo los endpoints de **/api/auth/**** son públicos; el resto requiere un token Bearer válido en cada petición.

La calidad del código se asegura con pruebas unitarias en JUnit 5, y el desarrollo se gestiona con Git y GitHub bajo la estrategia de Feature Branching. El build y la gestión de dependencias se realiza mediante Apache Maven 3.x.

Objetivos Principales

- Aplicar el paradigma de **Programación Orientada a Objetos** en un proyecto real y funcional.
- Implementar una **arquitectura en capas** con separación de responsabilidades (SRP de SOLID).
- Garantizar la **seguridad stateless** mediante JWT con Spring Security 6.
- Demostrar el uso de **patrones de diseño**: DTO, Repository, Front Controller, Chain of Responsibility.
- Validar la correcta integración entre capas mediante **pruebas unitarias** con JUnit 5.
- Gestionar el ciclo de vida del proyecto con **Git Feature Branching** e Issues de GitHub

2. Stack Tecnológico

Java 17 LTS Lenguaje base	Spring Boot 3.2.2 Framework principal	Spring Data JPAORM / Hibernate	Spring Security 6 Control de acceso
JWT 0.11.5 Tokens HS256	MapStruct 1.5.5 Mapeo de DTOs	MySQL 8.x Base de datos	Maven 3.x Build y dependencias
JUnit 5 Pruebas unitarias	Spring Security Test Testing de seguridad	Spring DevTools Recarga automática	Git & GitHub Feature Branching

Dependencias declaradas en pom.xml

Dependencia (artifactId)	Versión	Alcance	Propósito
spring-boot-starter-web	3.2.2	compile	API REST con Spring MVC
spring-boot-starter-data-jpa	3.2.2	compile	Persistencia con Hibernate
spring-boot-starter-security	3.2.2	compile	Autenticación y autorización
mysql-connector-j	8.x	runtime	Driver JDBC MySQL
jjwt-api / jjwt-impl / jjwt-jackson	0.11.5	compile/runtime	Generación y validación JWT (HS256)
mapstruct	1.5.5.Final	compile	Generación de mappers entidad-DTO
mapstruct-processor	1.5.5.Final	annotationProcessor	Procesador APT para código de mapeo
spring-boot-devtools	3.2.2	runtime/optional	Recarga automática en desarrollo
spring-boot-starter-test	3.2.2	test	JUnit 5 + Mockito
spring-security-test	3.2.2	test	Utilitarios de prueba para Spring Security

Nota sobre Lombok: Lombok 1.18.30 está declarado en el **pom.xml** pero **no se utiliza en las entidades JPA**. Spring Tool Suite 5.3.0 (Java 25 / OpenJDK) presenta incompatibilidad con Lombok como agente (**-javaagent**), por lo que los getters y setters se declaran manualmente. Esta decisión garantiza compilación limpia con el compilador ECJ.

3. Patrones de Diseño y Arquitectura de Capas

El proyecto implementa una **arquitectura multicapa** basada en la separación estricta de responsabilidades (principio SRP de SOLID). Cada paquete del proyecto representa una capa con un patrón de diseño asociado:

Capa / patrón	Implementación / detalle
🌐 controller — Patrón Front Controller	@RestController · ResponseEntity<T>
🔒 security — Chain of Responsibility · Stateless JWT	JwtFilter · JwtService · AppConfig · SecurityConfig
⚙️ service — Principio SRP · Lógica de Negocio	@Service · Interfaces + Implementaciones
📦 dto + mapper — Patrón DTO con MapStruct	Previene LazyInitializationException · Compilación automática
🗃️ repository — Patrón Repository	JpaRepository<T,ID> · Implementación automática Spring Data
📦 entity — Mapeo ORM · Sin Lombok (ECJ)	@Entity · @Table · Getters/Setters manuales
🗄️ MySQL 8.x — Base de datos relacional	irisdual_db · ddl-auto=update

Decisión de diseño: Sin Lombok en entidades

Las entidades JPA no utilizan Lombok. Spring Tool Suite 5.3.0 utiliza internamente Java 25 (OpenJDK), con la cual Lombok 1.18.30 no es compatible como agente (`-javaagent`). Los getters y setters se declaran manualmente.

Ejemplo en `Categoria.java`:

```
@Entity
@Table(name = "categorias")
public class Categoria {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "cat_id")
    private Integer id;

    @Column(name = "cat_nombre", nullable = false, length = 50)
    private String nombreCategoria;

    public Integer getId() { return id; }
    public void setId(Integer id) { this.id = id; }
    public String getNombreCategoria() { return nombreCategoria; }
    public void setNombreCategoria(String v) { this.nombreCategoria = v; }
}
```

Patrón DTO y MapStruct

`PiezaDTO` aplanar la relación `@ManyToOne(LAZY)` con `Categoria` evitando `LazyInitializationException` al serializar fuera de transacción JPA. La regla de mapeo en `PiezaMapper` es: `@Mapping(target = "nombreCategoria", source = "categoria.nombreCategoria")`. MapStruct genera el código de conversión en tiempo de compilación sin reflexión en runtime.

Arquitectura Stateless con JWT — Flujo completo

`SessionCreationPolicy.STATELESS` garantiza que el servidor no mantenga sesión HTTP. `JwtFilter` extiende `OncePerRequestFilter`, intercepta el header `Authorization: Bearer <token>`, extrae el email del subject, valida la firma HS256 y establece el contexto en `SecurityContextHolder`. Si el token es inválido o ausente, Spring Security deniega el acceso con 403 Forbidden.

La clase `AppConfig` (`@Configuration`) define cuatro beans críticos: `UserDetailsService` (carga usuario desde `IUsuarioRepository`), `BCryptPasswordEncoder` (hash de contraseñas), `DaoAuthenticationProvider` y `AuthenticationManager`.

Configuración del SecurityFilterChain

```
http
.csrf(csrf -> csrf.disable())
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/auth/**").permitAll()
    .anyRequest().authenticated()
)
.sessionManagement(session -> session
    .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
)
.authenticationProvider(authenticationProvider)
.addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
```

4. Diagrama UML de Clases

El siguiente diagrama modela la arquitectura completa del sistema: clases, interfaces, relaciones de herencia, implementación y asociaciones entre capas.

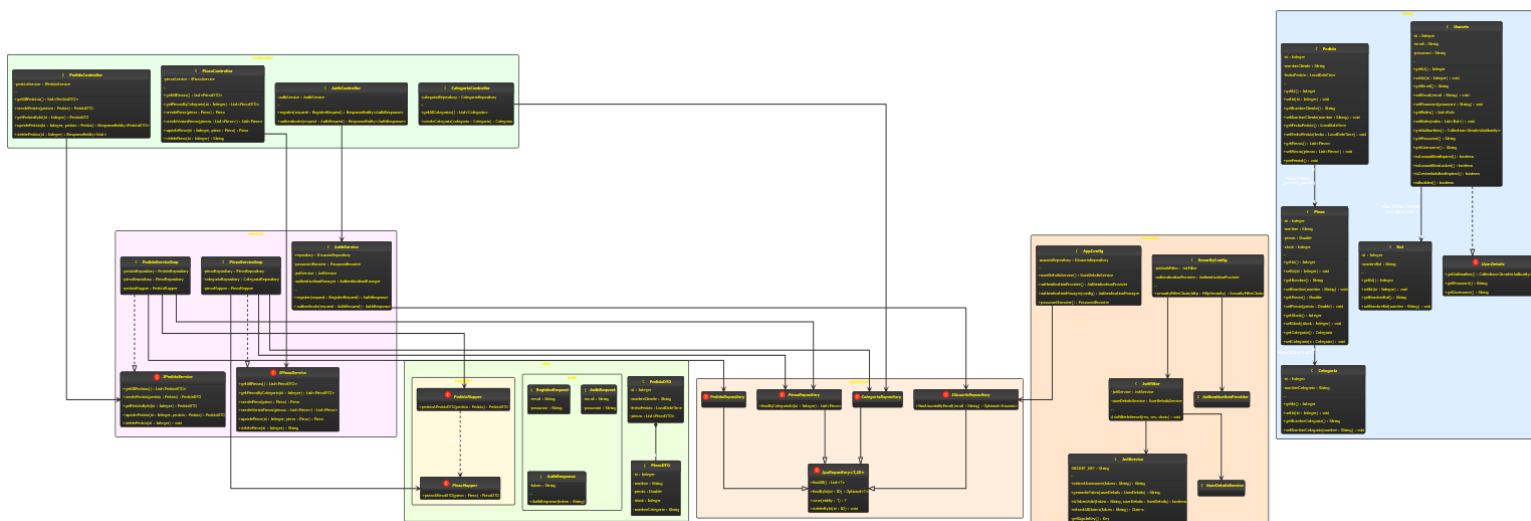


Figura 1 — Diagrama de Clases UML completo · IrisDual 3D Backend API

Leyenda de colores por capa

Azul — Entidades JPA	Verde — Repositorios	Amarillo — Servicios	Naranja — Controladores	Violeta — Seguridad
----------------------	----------------------	----------------------	-------------------------	---------------------

Capa	Clases / Interfaces	Patrón
Entidades	Categoria, Pieza, Pedido, Usuario, Rol	Active Record / ORM
Repositorios	CategoriaRepository, PiezaRepository, PedidoRepository, IUsuarioRepository	Repository
Servicios	IPiezaService, PiezaServiceImp, IPedidoService, PedidoServiceImp, AuthService	Service Layer / SRP
DTOs / Mappers	PiezaDTO, PedidoDTO, PiezaMapper, PedidoMapper, dto.auth.*	DTO Pattern
Controladores	CategoriaController, PiezaController, PedidoController, AuthController	Front Controller
Seguridad	JwtService, JwtFilter, AppConfig, SecurityConfig	Chain of Responsibility

5. Modelo de Datos y Persistencia Relacional

Hibernate genera automáticamente el esquema mediante `ddl-auto=update`. El modelo consta de cinco entidades con las siguientes relaciones:

Entidad	Tabla	Atributos Clave	Relación JPA
categoria	categorias	cat_id PK, cat_nombre NOT NULL (50)	—
Pieza	piezas	pieza_id, nombre, precio, stock	@ManyToOne(LAZY) → categorias
Pedido	pedidos	ped_id, nombre_cliente, fecha_pedido (@PrePersist)	@ManyToMany(LAZY) via pedidos_piezas
Rol	roles	id PK, nombre_rol	—
Usuario	usuarios	id, email, password (BCrypt hash)	@ManyToMany(EAGER) via usuarios_roles

Tablas intermedias generadas automáticamente

Tabla	FK A	FK B	Relación
pedidos_piezas	pedido_id	pieza_id	Pedido N:N Pieza
usuarios_roles	usuario_id	rol_id	Usuario N:N Rol

Configuración de conexión (application.properties)

```
spring.datasource.url=jdbc:mysql://localhost:3306/irisdual_db
spring.datasource.username=TU_USUARIO
spring.datasource.password=TU_CONTRASEÑA
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
spring.jpa.open-in-view=true
```

Diagrama Entidad-Relación (DER)

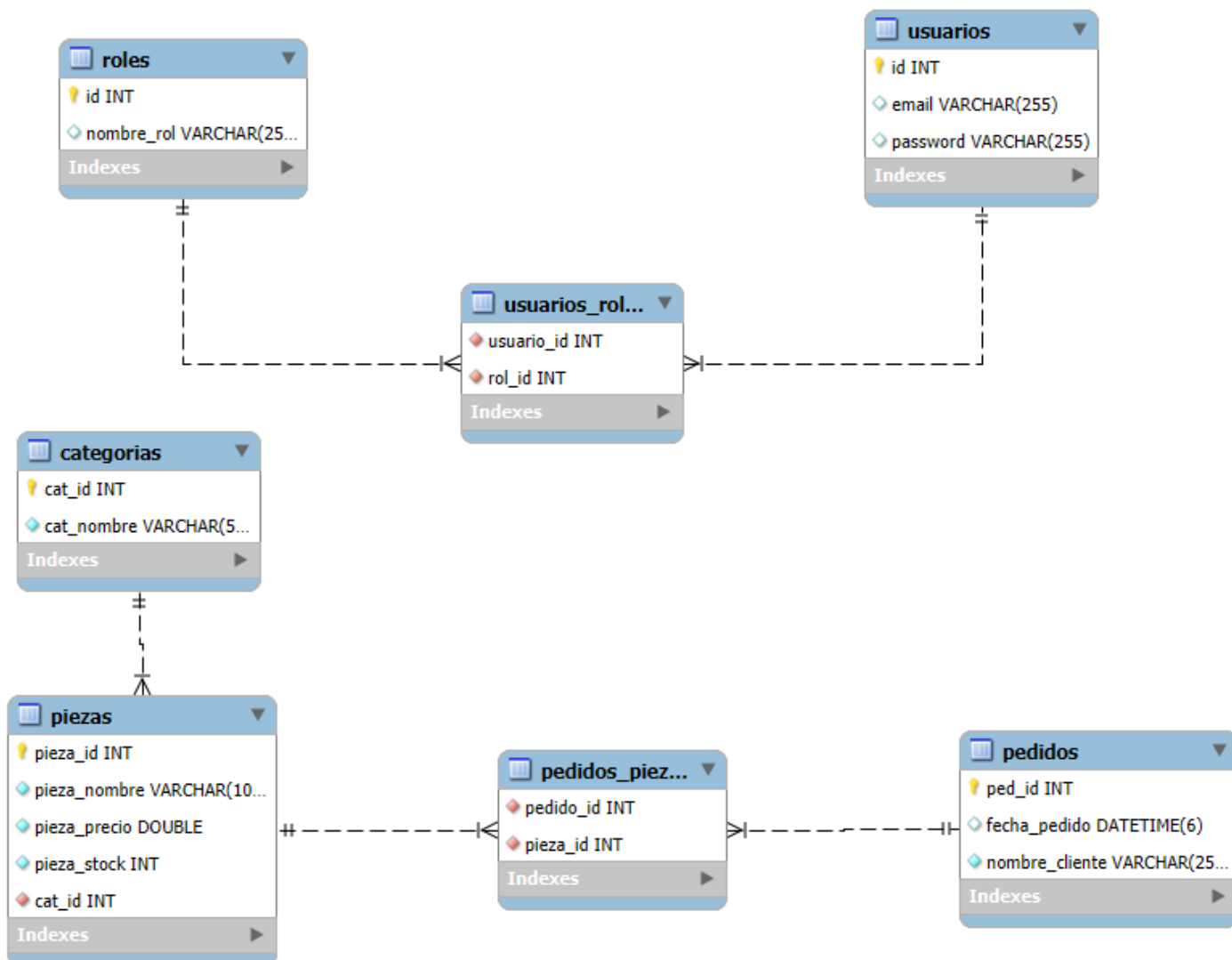


Figura 2 — Diagrama Entidad-Relación (DER) · Base de datos *irisduval_db*

6. Historias de Usuario y Gestión Ágil

El desarrollo fue organizado en cuatro Historias de Usuario, cada una con su rama en GitHub y criterios de aceptación verificados con Postman y JUnit 5.

Historia	Módulo	Horas est.	Iteración	Descripción breve
HU001	Persistencia de Catálogo	4 h	1	Registro de categorías y piezas; ORM sin Lombok; generación de esquema.
HU002	Listado y DTOs de Piezas	3 h	1	API de consulta del catálogo con patrón DTO + MapStruct.
HU003	Gestión de Pedidos y Excepciones	4 h	2	Pedidos N:M con piezas, manejo global de errores, pruebas assertThrows.
HU004	Seguridad JWT	6 h	2	Autenticación stateless; JwtFilter; SecurityFilterChain; roles de usuario.

Código: HU001	Usuario: Administrador
Nombre de la Historia de Usuario: Persistencia de Catálogo	
Prioridad: Alta	Riesgo de desarrollo: Bajo
Estimación: 4 horas	iteración Asignada: 1
Responsable: Walter Leonel Cussi	
Descripción: Como admin quiero registrar categorías y piezas para mantener el catálogo.	
Enfoque/Objetivos: Modelar el ORM sin Lombok y generar el esquema en BD. Tareas: 1. Modelar Categoría y Pieza con getters/setters manuales 2. Configurar Hibernate (ddl-auto=update). 3. Crear repositorios JpaRepository. Criterios de aceptación: 1. Tablas generadas automáticamente en irisdual_db. 2. Prueba unitaria de inyección superada. División de tareas: Base de Datos / Backend	
Observaciones: Usar getters/setters manuales por compatibilidad con ECJ (STS 5.3.0). ddl-auto=update genera tablas al arrancar la app.	

Código: HU002	Usuario: Cliente
Nombre de la Historia de Usuario: Listado y DTOs de Piezas	
Prioridad: Alta	Riesgo de desarrollo: Bajo
Estimación: 3 horas	iteración Asignada: 1
Responsable: Walter Leonel Cussi	
Descripción: Como cliente quiero consultar el catálogo viendo nombre, precio, stock y categoría.	
Enfoque/Objetivos: Implementar patrón DTO para desacoplar el modelo físico de la presentación. Tareas: 1. Crear clases DTO (PiezaDTO, PedidoDTO). 2. Configurar MapStruct con @Mapping para relaciones LAZY. 3. Desarrollar PiezaController con endpoints GET Criterios de aceptación: 1. El endpoint devuelve JSON sin exponer el modelo físico. 2. Sin errores de LazyInitializationException. 3. Postman recibe 200 OK con datos correctos. División de tareas: Backend / API REST	
Observaciones: El mapeo de categoria.nombreCategoria en PiezaDTO previene recursividad y serialización fallida de entidades con relaciones LAZY.	

Código: HU003	Usuario: Cliente
Nombre de la Historia de Usuario: Gestión de Pedidos y Excepciones	
Prioridad: Alta	Riesgo de desarrollo: Medio
Estimación: 4 horas	iteración Asignada: 2
Responsable: Walter Leonel Cussi	
Descripción: Como cliente quiero generar pedidos con múltiples piezas y ver errores claros si algo falla.	
Enfoque/Objetivos: Resolver relación N:M transaccional y capturar errores globales con mensajes semánticos. Tareas: 1. Modelar Pedido con relación @ManyToMany a Pieza. 2. Implementar PedidoServiceImp con @Transactional. 3. Crear ControllerAdvice para manejo global de excepciones. 4. Escribir pruebas con assertThrows para casos de error. Criterios de aceptación: 1. Se insertan datos en tabla intermedia pedidos_piezas. 2. El sistema devuelve 404 semántico si la pieza no existe. 3. @PrePersist establece fecha_pedido automáticamente. División de tareas: Base de Datos / Backend	
Observaciones: Servicio anotado con @Transactional a nivel de clase para garantizar contexto JPA activo en toda la operación.	

Código: HU004	Usuario: Administrador
Nombre de la Historia de Usuario: Gestión de Pedidos y Excepciones	
Prioridad: Alta	Riesgo de desarrollo: Alto
Estimación: 6 horas	iteración Asignada: 2
Responsable: Walter Leonel Cussi	
Descripción: Como admin quiero restringir la API para que solo usuarios registrados operen.	
Enfoque/Objetivos: Implementar autenticación stateless con filtros HTTP y JWT. Tareas: 1. Modelar Usuario y Rol implementando UserDetails. 2. Crear JwtService con JWT 0.11.5 (HS256, 24 h). 3. Implementar JwtFilter extendiendo OncePerRequestFilter. 4. Configurar SecurityFilterChain con lambda DSL de Spring Security 6. Criterios de aceptación: 1. POST /api/auth/register y /authenticate retornan 200 OK con token. 2. Endpoints protegidos devuelven 403 Forbidden sin token Bearer. 3. Endpoints protegidos retornan 200 OK con token válido. División de tareas: Seguridad / Backend	
Observaciones: JwtFilter extiende OncePerRequestFilter para ejecución única por petición. Clave secreta HS256 de 256 bits en Base64. Contraseñas hasheadas con BCryptPasswordEncoder.	

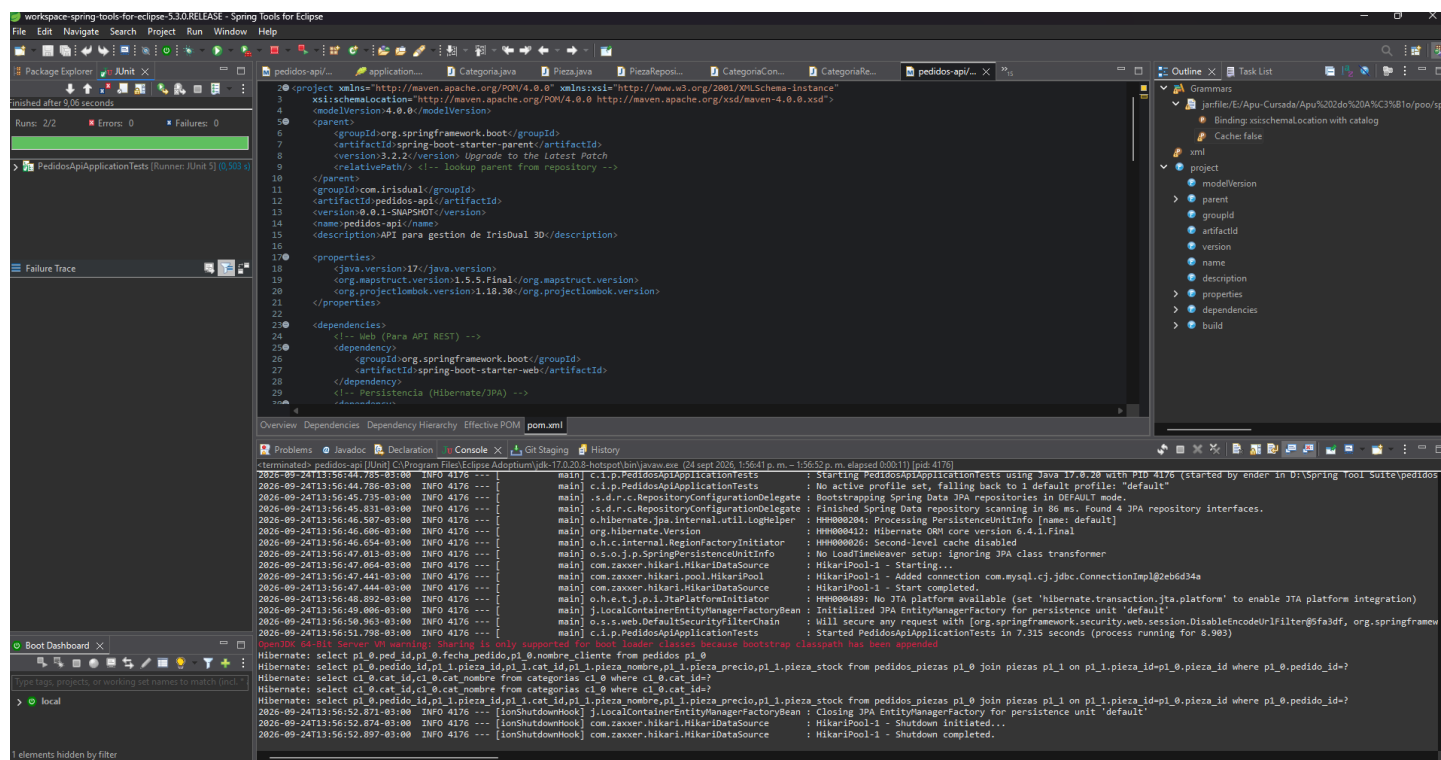
7. Aseguramiento de Calidad — Testing con JUnit 5

Las pruebas unitarias implementadas con JUnit 5 verifican el correcto funcionamiento del contexto de Spring, la inyección de dependencias, la integridad de transacciones JPA y el comportamiento ante entradas inválidas.

Pruebas implementadas en PedidosApiApplicationTests

Método de prueba	Descripción	Assertion	Resultado
verificarInyeccionRepositorio()	Verifica que el contexto de Spring levanta correctamente y el repositorio de pedidos está inyectado por @Autowired.	assertNotNull(pedidoRepository)	✓ PASS
verificarConsultasPedido()	Valida que la consulta general de pedidos se ejecuta sin lanzar excepciones de Hibernate (LazyInitializationException, etc.).	assertDoesNotThrow(() -> pedidoService.getAllPedidos())	✓ PASS

- Carga correcta del contexto de aplicación (@SpringBootTest)
- Correcta inyección de repositorios y servicios vía @Autowired
- Ausencia de errores de serialización JPA en consultas reales
- Propagación de excepciones desde la capa de servicio hacia el controlador



```

2026-09-24T13:56:44.785-03:00 INFO 4176 --- main c.i.p.PedidosApiApplicationTests : Starting PedidosApiApplicationTests using Java 17.0.20 with PID 4176 (started by ender in U:\Spring Tool Suite\pedidos-api)
2026-09-24T13:56:44.786-03:00 INFO 4176 --- main c.i.p.PedidosApiApplicationTests : No active profile set, falling back to 1 default profile: "default"
2026-09-24T13:56:45.735-03:00 INFO 4176 --- main s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2026-09-24T13:56:45.831-03:00 INFO 4176 --- main s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 86 ms. Found 4 JPA repository interfaces.
2026-09-24T13:56:46.507-03:00 INFO 4176 --- main o.hiber.hjpa.internal.util.LogHelper : HHH00000412: Hibernate ORM core version 6.4.1.Final
2026-09-24T13:56:46.686-03:00 INFO 4176 --- main org.hibernate.Version : HHH0000026: Second-level cache disabled
2026-09-24T13:56:46.654-03:00 INFO 4176 --- main o.s.o.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver setup; ignoring JPA class transformer
2026-09-24T13:56:47.013-03:00 INFO 4176 --- main com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
2026-09-24T13:56:47.441-03:00 INFO 4176 --- main com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Added connection com.mysql.cj.jdbc.ConnectionImpl@2eb6d34e
2026-09-24T13:56:47.444-03:00 INFO 4176 --- main com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
2026-09-24T13:56:48.892-03:00 INFO 4176 --- main o.h.e.t.j.p.i.JtaPlatformInitiator : HHH0000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform integration)
2026-09-24T13:56:49.006-03:00 INFO 4176 --- main j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-09-24T13:56:50.963-03:00 INFO 4176 --- main o.s.s.web.DefaultSecurityFilterChain : Will secure any request with [org.springframework.security.web.session.DisableEncodeUrlFilter@5fa3df, org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter]
2026-09-24T13:56:51.798-03:00 INFO 4176 --- main c.i.p.PedidosApiApplicationTests : Started PedidosApiApplicationTests in 7.315 seconds (process running for 8.903)
2026-09-24T13:56:52.897-03:00 INFO 4176 --- [main] c.i.p.PedidosApiApplicationTests : Hibernate: select p1_0.ped_id,p1_0.fecha_pedido,p1_0.nombre_cliente from pedidos p1_0
2026-09-24T13:56:52.897-03:00 INFO 4176 --- [main] c.i.p.PedidosApiApplicationTests : Hibernate: select p1_0.ped_id,p1_1.pieza_id,p1_1.cat_id,p1_1.pieza_nombre,p1_1.pieza_precio,p1_1.pieza_stock from pedidos_piezas p1_0 join piezas p1_1 on p1_1.pieza_id=p1_0.pieza_id where p1_0.ped_id=?
2026-09-24T13:56:52.897-03:00 INFO 4176 --- [main] c.i.p.PedidosApiApplicationTests : Hibernate: select c1_0.cat_id,c1_0.cat_nombre from categorias c1_0 where c1_0.cat_id=?
2026-09-24T13:56:52.897-03:00 INFO 4176 --- [main] c.i.p.PedidosApiApplicationTests : Hibernate: select p1_0.ped_id,p1_1.pieza_id,p1_1.cat_id,p1_1.pieza_nombre,p1_1.pieza_precio,p1_1.pieza_stock from pedidos_piezas p1_0 join piezas p1_1 on p1_1.pieza_id=p1_0.pieza_id where p1_0.ped_id=?
2026-09-24T13:56:52.897-03:00 INFO 4176 --- [main] c.i.p.PedidosApiApplicationTests : HikariPool-1 - Shutdown initiated...
2026-09-24T13:56:52.897-03:00 INFO 4176 --- [main] c.i.p.PedidosApiApplicationTests : HikariPool-1 - Shutdown completed.
  
```

Figura 3 — Resultados de ejecución de pruebas unitarias con JUnit 5 · Spring Tool Suite

Resultado: Todas las pruebas definidas pasaron exitosamente. El contexto de Spring levanta sin errores y las validaciones responden según los criterios de aceptación de cada Historia de Usuario. El uso de @Transactional a nivel de clase en PedidoServiceImp garantiza que las operaciones JPA con relaciones LAZY se ejecuten dentro de un contexto de sesión válido.

8. Endpoints, Contratos de API y Evidencias de Seguridad

Autenticación — Rutas Públicas (/api/auth/**)

POST	/api/auth/register	público	Registra usuario y devuelve token JWT
POST	/api/auth/authenticate	público	Login — devuelve token JWT

Body de request (JSON) para ambos endpoints:

```
{
  "email": "usuario@ejemplo.com",
  "password": "contraseña123"
}
```

Respuesta esperada — AuthResponse:

```
{
  "token":
  "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c3Vhcm1vQG9qZW1wbG8uY29tIi4uLn0.FIRMA" }

```

Categorías — Requieren Bearer Token

GET	/api/categorias	protegido	Lista todas las categorías
POST	/api/categorias	protegido	Crea nueva categoría

Piezas — Requieren Bearer Token

GET	/api/piezas	protegido	Lista piezas (retorna PiezaDTO)
GET	/api/piezas/categoria/{id}	protegido	Filtra piezas por categoría
POST	/api/piezas	protegido	Crea una pieza
POST	/api/piezas/lote	protegido	Carga masiva de piezas (batch)
PUT	/api/piezas/{id}	protegido	Actualiza pieza existente
DELETE	/api/piezas/{id}	protegido	Elimina pieza · 204 No Content

Pedidos — Requieren Bearer Token

GET	/api/pedidos	protegido	Lista pedidos (retorna PedidoDTO)
GET	/api/pedidos/{id}	protegido	Obtiene pedido por ID
POST	/api/pedidos	protegido	Crea pedido con lista de piezas
PUT	/api/pedidos/{id}	protegido	Actualiza pedido y sus piezas
DELETE	/api/pedidos/{id}	protegido	Elimina pedido · 204 No Content

Uso del token JWT: Incluir el token en el header de cada petición protegida:

Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...

Evidencias en Postman



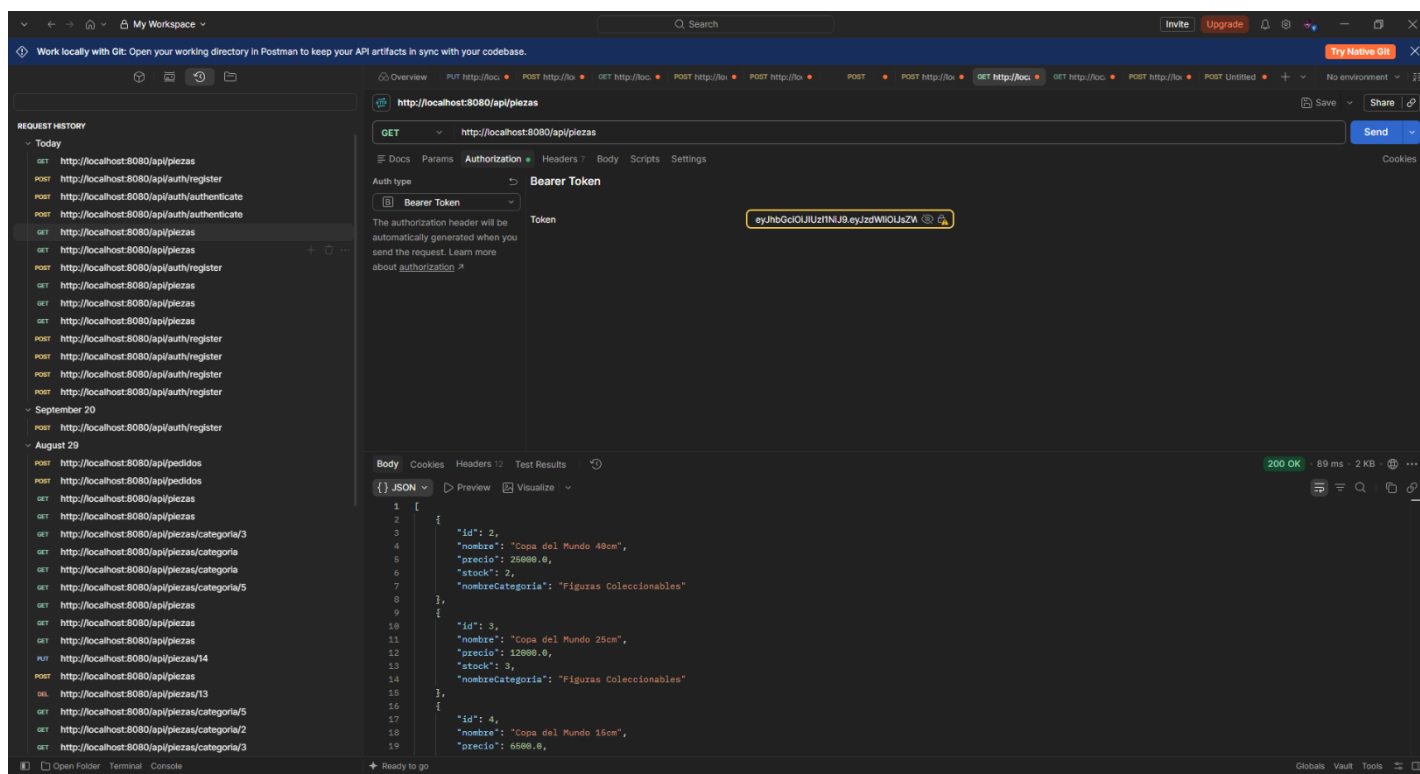


Figura 6 — Endpoint con Authorization: Bearer válido · Respuesta 200 OK

9. Gestión de Versiones — Git & GitHub

El proyecto adoptó la estrategia de Feature Branching. Cada Historia de Usuario fue desarrollada en una rama dedicada, garantizando que `main` siempre contenga código estable y aprobado mediante Pull Request.

Rama	Propósito	Criterio de merge
<code>main</code>	Rama principal estable y entregable.	Solo recibe merges de features completadas y aprobadas.
<code>feature/hu001</code>	Persistencia de catálogo — entidades y repositorios	Tablas generadas + prueba unitaria OK.
<code>feature/hu002</code>	DTOs y endpoints de piezas.	200 OK en Postman sin LazyInit.
<code>feature/hu003</code>	Pedidos con relación N:M y manejo de excepciones.	INSERT en pedidos_piezas + 404 semántico.
<code>feature/hu004</code>	Seguridad JWT stateless.	403 sin token + 200 con token Bearer.

- Cada commit está referenciado a su Issue de GitHub para trazabilidad completa.
- Los Pull Requests documentan las decisiones técnicas de cada iteración.
- El tablero Kanban de GitHub Projects refleja el estado de cada Historia de Usuario.

Beneficio del Feature Branching: Esta estrategia permite desarrollo aislado por funcionalidad, facilita la revisión de código y asegura que cada entrega esté alineada con una Historia de Usuario verificada con sus criterios de aceptación.

Network Graph — Ramas de desarrollo

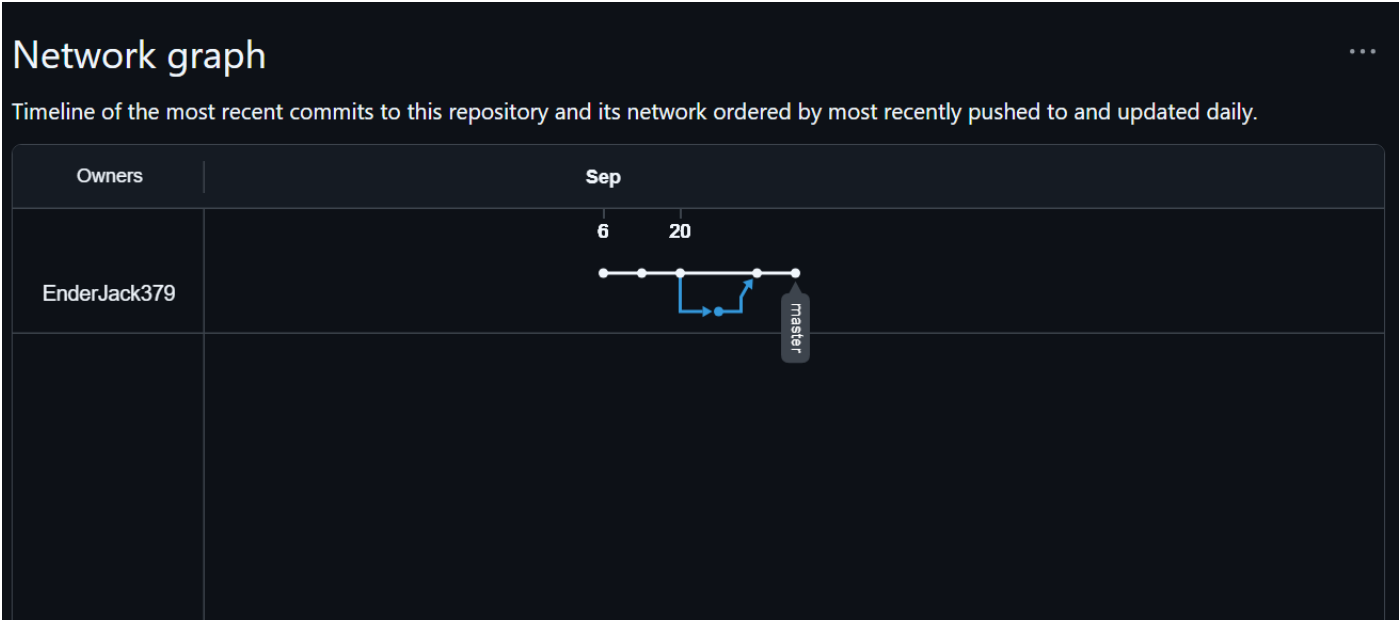


Figura 7 — Network Graph · Historial de ramas por Historia de Usuario

Trazabilidad mediante Issues

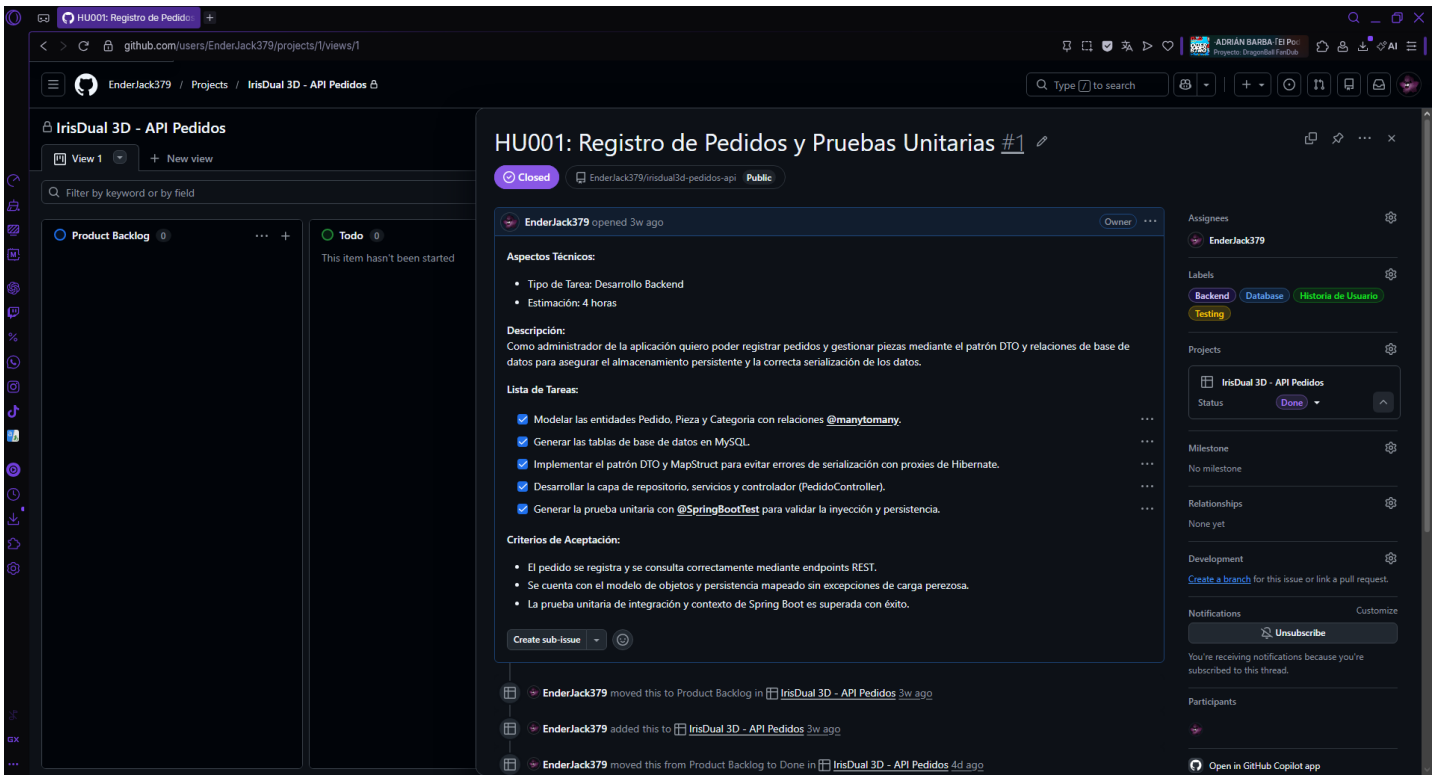


Figura 8 — Issues de GitHub · Trazabilidad entre tareas, commits y criterios de aceptación

Tablero Kanban de gestión

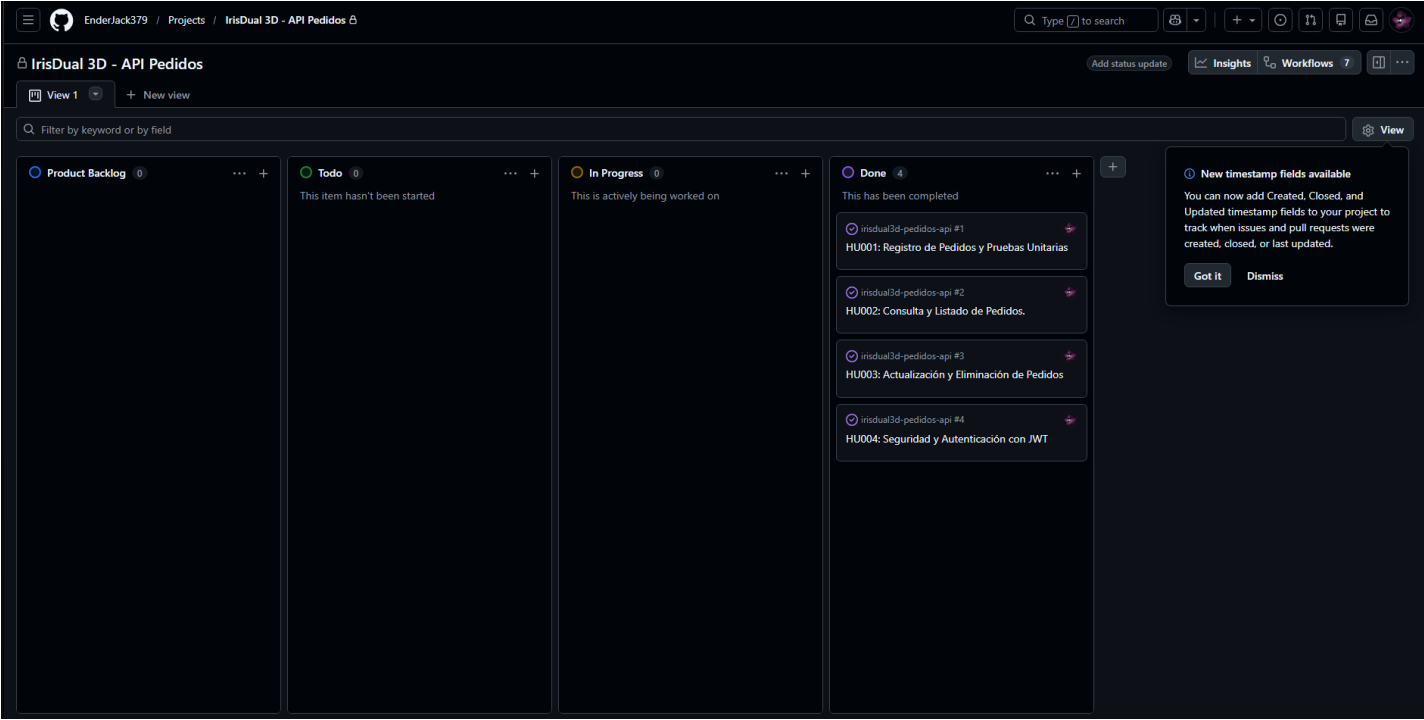


Figura 9 — GitHub Projects · Tablero Kanban de seguimiento de Historias de Usuario

10. Estructura de Archivos y Organización del Proyecto

A continuación, se detalla la estructura de paquetes, clases Java, recursos y configuraciones que conforman la arquitectura backend de IrisDual 3D API (excluyendo artefactos de compilación):

pedidos-api/	
├ pom.xml	# Dependencias Maven (Spring Boot 3.2.2, MapStruct, JWT)
├ src/	
│ └ main/	
│ │ └ java/com/irisdual/pedidos_api/	
│ │ │ └ PedidosApiApplication.java	# Clase de inicio (@SpringBootApplication)
│ │ │ └ config/SecurityConfig.java	# Placeholder (reemplazado por security/SecurityConfig)
│ │ │ └ controller/	# Endpoints REST API
│ │ │ │ └ AuthController.java	# POST /api/auth/register y /authenticate
│ │ │ │ └ CategoriaController.java	# CRUD de Categorías
│ │ │ │ └ PedidoController.java	# Gestión de Pedidos (CRUD completo)
│ │ │ │ └ PiezaController.java	# Gestión de Piezas 3D (CRUD + batch)
│ │ │ └ dto/	# Objetos de Transferencia de Datos
│ │ │ │ └ PedidoDTO.java	
│ │ │ │ └ PiezaDTO.java	# Aplanar relación LAZY con Categoría
│ │ │ │ └ auth/	
│ │ │ │ │ └ AuthRequest.java	# {email, password} - Login
│ │ │ │ │ └ AuthResponse.java	# {token} - Respuesta JWT
│ │ │ │ │ └ RegisterRequest.java	# {email, password} - Registro
│ │ │ │ └ mapper/	
│ │ │ │ │ └ PedidoMapper.java	# Generado por MapStruct en compilación
│ │ │ │ │ └ PiezaMapper.java	# @Mapping categoria.nombreCategoria
│ │ │ └ entity/	# Entidades JPA - Sin Lombok
│ │ │ │ └ Categoria.java	# @Entity - cat_id PK
│ │ │ │ └ Pedido.java	# @ManyToMany con Pieza - @PrePersist
│ │ │ │ └ Pieza.java	# @ManyToOne(LAZY) con Categoría
│ │ │ │ └ Rol.java	# Entidad Rol
│ │ │ │ └ Usuario.java	# Implementa UserDetails
│ │ │ └ repository/	# Repositorios Spring Data JPA
│ │ │ │ └ CategoriaRepository.java	# JpaRepository<Categoria, Integer>
│ │ │ │ └ IUserRepository.java	# findUsuarioByEmail -> Optional
│ │ │ │ └ PedidoRepository.java	
│ │ │ │ └ PiezaRepository.java	
│ │ │ └ security/	# Seguridad Stateless y JWT
│ │ │ │ └ AppConfig.java	# UserDetailsService - BCrypt - AuthProvider
│ │ │ │ └ JwtFilter.java	# OncePerRequestFilter - valida token
│ │ │ │ └ JwtService.java	# Genera/valida tokens HS256 - 24 h
│ │ │ │ └ SecurityConfig.java	# SecurityFilterChain - STATELESS
│ │ │ └ service/	# Capa de Lógica de Negocio
│ │ │ │ └ AuthService.java	# register() y authenticate()
│ │ │ │ └ IPedidoService.java	# Interfaz del servicio de pedidos
│ │ │ │ └ IPiezaService.java	# Interfaz del servicio de piezas
│ │ │ │ └ PedidoServiceImp.java	# @Transactional - valida piezas
│ │ │ │ └ PiezaServiceImp.java	
│ └ resources/	
│ │ └ application.properties	# Config MySQL, Hibernate y JWT Secret
└ test/java/com/irisdual/pedidos_api/	
└ └ PedidosApiApplicationTests.java	# 2 tests JUnit 5

Capa / Paquete	Responsabilidad Principal	Tecnología Clave
controller	Atención de peticiones HTTP, validación de parámetros y retorno de respuestas JSON.	Spring Web MVC (@RestController)
service	Procesamiento de la lógica de negocio, manejo de transacciones y excepciones.	Spring Core (@Service, @Transactional)
repository	Operaciones CRUD e interfaz directa con la base de datos MySQL.	Spring Data JPA (JpaRepository)
entity	Definición del esquema relacional y mapeo de tablas en la BD. Sin Lombok.	Jakarta Persistence API (JPA / Hibernate)
dto & mapper	Desacoplamiento del modelo físico respecto a la capa de presentación.	MapStruct 1.5.5
security	Gestión de credenciales, autenticación JWT stateless y encriptación de contraseñas.	Spring Security 6 & JWT 0.11.5



11. Guía de Instalación y Ejecución

A continuación, se describen los pasos necesarios para clonar, configurar y ejecutar la API en un entorno local.

Prerrequisitos

- **Java 17** instalado y configurado en el **PATH** del sistema.
- **MySQL 8.x** corriendo localmente (puerto 3306 por defecto).
- **Maven 3.x** (o usar el wrapper incluido `./mvnw`).
- IDE recomendado: Spring Tool Suite 4/5 o IntelliJ IDEA.

Pasos de instalación

1. Clonar el repositorio

```
git clone https://github.com/EnderJack379/irisdual3d-pedidos-api.git cd irisdual3d-pedidos-api
```

2. Crear la base de datos en MySQL

```
CREATE DATABASE irisdual_db;
```

3. Configurar la conexión en `application.properties`

```
spring.datasource.url=jdbc:mysql://localhost:3306/irisdual_db spring.datasource.username=TU_USUARIO  
spring.datasource.password=TU_CONTRASEÑA spring.jpa.show-sql=true spring.jpa.hibernate.ddl-auto=update  
spring.jpa.open-in-view=true
```

4. Compilar e instalar dependencias

```
./mvnw clean install
```

5. Levantar la aplicación

```
./mvnw spring-boot:runLa
```

API quedará disponible en: `http://localhost:8080`

Flujo de uso básico

1. Registrar un usuario: `POST /api/auth/register` con `{"email":"...", "password":"..."}`
2. Obtener el token JWT de la respuesta: `{"token": "eyJ..."}`
3. Usar el token en el header de cada petición: `Authorization: Bearer <token>`
4. Consumir los endpoints de categorías, piezas y pedidos.

Importante: El token JWT tiene una expiración de 24 horas. Pasado ese tiempo, el servidor devuelve `403 Forbidden` y es necesario autenticarse nuevamente.



12. Conclusiones

El desarrollo de IrisDual 3D Backend API como Trabajo Práctico Integrador permitió aplicar en un proyecto real y funcional los conceptos fundamentales de la Programación Orientada a Objetos y los patrones de diseño estudiados durante la cursada. El resultado es una API REST completamente operativa, segura y organizada bajo principios SOLID.

Logros técnicos alcanzados

- **Arquitectura en capas** con separación clara de responsabilidades: controller → service → repository → entity.
- **Autenticación stateless** con JWT (HS256, 24 h) usando Spring Security 6 en modo lambda DSL moderno, sin WebSecurityConfigurerAdapter.
- **Patrón DTO + MapStruct** que previene LazyInitializationException y desacopla el modelo físico de la presentación.
- **Relaciones JPA complejas:** @ManyToOne y @ManyToMany con tabla intermedia generada automáticamente por Hibernate.
- **Testing con JUnit 5:** verificación de inyección de dependencias y consultas sin errores de Hibernate.
- **Feature Branching** en Git con trazabilidad completa mediante Issues y tablero Kanban en GitHub Projects.
- **Decisión fundamentada** de no usar Lombok en entidades, garantizando compatibilidad con ECJ de Spring Tool Suite 5.3.0.

Aprendizajes clave

- La importancia de la **gestión de transacciones JPA (@Transactional)** para evitar problemas con la carga perezosa de relaciones.
- El valor del **patrón DTO** para no exponer el modelo interno de la base de datos directamente en la API.
- La complejidad real de implementar **seguridad stateless** con filtros personalizados y la cadena de responsabilidad de Spring Security.
- La importancia de las **pruebas unitarias** para validar el comportamiento del sistema de manera reproducible y confiable.

Estado del proyecto: La API se encuentra completamente funcional, con todos los endpoints operativos, seguridad JWT implementada, pruebas unitarias superadas y código gestionado mediante Feature Branching en GitHub. Todas las Historias de Usuario fueron completadas con sus criterios de aceptación verificados.